Practice 08 — Build a Two-Tier VPC

Objective: build your own network: a public subnet for a web server and a private subnet for an app server that the internet can't reach. Prove it works, break it on purpose, fix it, and delete everything. These are the skills from today's lecture.

Timebox: ~45–50 min, in class. If you're stuck on one step for more than 10 minutes, ask instead of pushing through alone.

> **How you're checked:** this practice is done in class and **checked orally** at the end. There's no written report. Keep your terminal (and console tabs) open, be ready to **show** each item in the Definition of done and **explain** what each step does. The instructor will ask you a few of the questions under "Be ready to explain".

> **Two tracks.** Do **Track A** in **AWS Academy Learner Lab**. If you can't get into Learner Lab yet, do **Track B**, which builds the same shape with Linux network namespaces (Linux VM or WSL only). Tell the instructor which track you did.

> **Using AWS Academy Learner Lab (Track A)**
>
> 1. In AWS Academy: **Modules** → **Launch AWS Academy Learner Lab** → **Start Lab**. Wait until the dot next to "AWS" turns **green**, then click **AWS** to open the console.
> 2. Work in region `us-east-1` (N. Virginia). Per the Learner Lab Readme, only `us-east-1` and `us-west-2` are allowed, and creating VPCs is allowed.
> 3. For the CLI, open **AWS Details** → **AWS CLI** → **Show** and paste the credentials into `~/.aws/credentials`. They are **temporary** and include a session token, so copy them again each session. Or use the terminal on the Learner Lab page, which is already set up.
> 4. Use the ready-made key pair `vockey`: **AWS Details** → **Download PEM** gives you `labsuser.pem`.
> 5. When you're done: clean up (A8), then click **End Lab**.

Definition of done (things you can show)

□ Your VPC with **two subnets**, and the two route tables: public (`0.0.0.0/0 → igw`) and private (`local` only)
□ The web page served from the **public** subnet
□ An SSH session **from the web server into the app server**
□ Proof the app server is private: no public IP, `curl` to the internet times out, and you can't reach it from your laptop
□ The break-and-fix: page dead without the `0.0.0.0/0` route, alive again after re-adding it

□ Cleanup: no VPC, instances, security groups or IGW of yours left (Track A: and the lab is ended)

Track A — AWS Academy Learner Lab

Start the lab first (see the box above). Keep **one terminal** open for the whole practice: the steps reuse shell variables (`$VPC` , `$PUB` , …). If you lose them, look the IDs up in the console.

```
export AWS_REGION=us-east-1
aws sts get-caller-identity          # Arn ends in assumed-role/voclabs/...
```

A1. VPC and subnets

```
VPC=$(aws ec2 create-vpc --cidr-block 10.0.0.0/16 \
  --tag-specifications 'ResourceType=vpc,Tags=[{Key=Name,Value=lab08-vpc}]' \
  --query Vpc.VpcId --output text)
aws ec2 modify-vpc-attribute --vpc-id $VPC --enable-dns-hostnames Value=true

AZ=$(aws ec2 describe-availability-zones \
  --query 'AvailabilityZones[0].ZoneName' --output text)

PUB=$(aws ec2 create-subnet --vpc-id $VPC --cidr-block 10.0.1.0/24 \
  --availability-zone $AZ \
  --tag-specifications 'ResourceType=subnet,Tags=[{Key=Name,Value=lab08-public}]' \
  --query Subnet.SubnetId --output text)
PRIV=$(aws ec2 create-subnet --vpc-id $VPC --cidr-block 10.0.2.0/24 \
  --availability-zone $AZ \
  --tag-specifications 'ResourceType=subnet,Tags=[{Key=Name,Value=lab08-private}]' \
  --query Subnet.SubnetId --output text)

# instances launched in the public subnet get a public IP automatically
aws ec2 modify-subnet-attribute --subnet-id $PUB --map-public-ip-on-launch
echo "$VPC $AZ $PUB $PRIV"
```

A2. Internet gateway and the public route table

```
IGW=$(aws ec2 create-internet-gateway \
  --query InternetGateway.InternetGatewayId --output text)
aws ec2 attach-internet-gateway --internet-gateway-id $IGW --vpc-id $VPC

RT=$(aws ec2 create-route-table --vpc-id $VPC \
  --tag-specifications 'ResourceType=route-table,Tags=[{Key=Name,Value=lab08-public-rt}]' \
  --query RouteTable.RouteTableId --output text)
aws ec2 create-route --route-table-id $RT \
  --destination-cidr-block 0.0.0.0/0 --gateway-id $IGW
ASSOC=$(aws ec2 associate-route-table --route-table-id $RT --subnet-id $PUB \
  --query AssociationId --output text)

aws ec2 describe-route-tables --filters Name=vpc-id,Values=$VPC \
  --query 'RouteTables[].[RouteTableId,Associations[0].Main,Routes[].DestinationCidrBlock]' \
  --output json
```

The private subnet has no explicit association, so it uses the VPC's **main** route table, which only has the `local` route. That's what makes it private. We don't add a NAT gateway: it's billed per hour, and the app server doesn't need the internet today.

A3. Security groups

```
MYIP=$(curl -s https://checkip.amazonaws.com)

SG_WEB=$(aws ec2 create-security-group --group-name lab08-web \
  --description "web tier" --vpc-id $VPC --query GroupId --output text)
aws ec2 authorize-security-group-ingress --group-id $SG_WEB \
  --protocol tcp --port 22 --cidr $MYIP/32
aws ec2 authorize-security-group-ingress --group-id $SG_WEB \
  --protocol tcp --port 80 --cidr 0.0.0.0/0

SG_APP=$(aws ec2 create-security-group --group-name lab08-app \
  --description "app tier" --vpc-id $VPC --query GroupId --output text)
# SSH and ping only from instances in the web security group
aws ec2 authorize-security-group-ingress --group-id $SG_APP \
  --protocol tcp --port 22 --source-group $SG_WEB
aws ec2 authorize-security-group-ingress --group-id $SG_APP \
  --protocol icmp --port -1 --source-group $SG_WEB
```

A4. Launch the two instances

Save as `web-data.sh` (put **your name** in it):

```
#!/bin/bash
dnf install -y nginx
echo "<h1>CSS4003 — Your Name — public subnet</h1>" > /usr/share/nginx/html/index.html
systemctl enable --now nginx
```

```
AMI=resolve:ssm:/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64

WEB=$(aws ec2 run-instances --image-id $AMI --instance-type t3.micro \
  --key-name vockey --subnet-id $PUB --security-group-ids $SG_WEB \
  --user-data file://web-data.sh \
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=lab08-web}]' \
  --query 'Instances[0].InstanceId' --output text)

APP=$(aws ec2 run-instances --image-id $AMI --instance-type t3.micro \
  --key-name vockey --subnet-id $PRIV --security-group-ids $SG_APP \
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=lab08-app}]' \
  --query 'Instances[0].InstanceId' --output text)

aws ec2 wait instance-running --instance-ids $WEB $APP
aws ec2 describe-instances --instance-ids $WEB $APP \
  --query 'Reservations[].Instances[].[Tags[?Key==`Name`]|
[0].Value,SubnetId,PrivateIpAddress,PublicIpAddress]' \
  --output table
```

The app instance's public IP column is **empty** (`None`). Note both private IPs. `t3.micro` works in Learner Lab; if a launch is refused, use `t2.micro` .

A5. Web page, then hop to the app server

```
WEB_IP=<public IP of lab08-web>
APP_IP=<private IP of lab08-app, 10.0.2.x>
curl http://$WEB_IP          # wait 1–2 min for cloud-init to install nginx
```

Load the key into your SSH agent and forward it, so the key never leaves your laptop:

```
chmod 400 labsuser.pem
ssh-add labsuser.pem
ssh -A ec2-user@$WEB_IP
# now on the web server:
ping -c 2 10.0.2.x            # app's private IP: works (local route + SG-app)
ssh ec2-user@10.0.2.x         # works: SG-app allows 22 from SG-web
```

> If `ssh-add` fails (common on Windows when the ssh-agent service is off), copy the key to the web server instead: `scp -i labsuser.pem labsuser.pem ec2-user@$WEB_IP:~` and on the web server `chmod 400 labsuser.pem && ssh -i labsuser.pem ec2-user@10.0.2.x`. Delete the copy afterwards (`rm labsuser.pem`).

A6. Prove the app server is private

On the **app** server (through the web server):

```
curl -m 5 -I https://aws.amazon.com     # times out: no route to the internet
exit                                    # back to web
curl -m 5 -I https://aws.amazon.com     # works on web: public subnet
exit                                    # back to your laptop
```

On your **laptop**:

```
ssh -o ConnectTimeout=5 ec2-user@10.0.2.x   # fails: a private address isn't reachable from the
internet
```

A7. Break it, then fix it

```
aws ec2 delete-route --route-table-id $RT --destination-cidr-block 0.0.0.0/0
curl -m 5 http://$WEB_IP          # now times out — same server, same SG, no way out

aws ec2 create-route --route-table-id $RT \
  --destination-cidr-block 0.0.0.0/0 --gateway-id $IGW
curl -m 5 http://$WEB_IP          # works again
```

Be ready to explain why the security group didn't help while the route was missing.

A8. Cleanup (in this order)

Dependencies decide the order: nothing can be deleted while something still uses it.

```
aws ec2 terminate-instances --instance-ids $WEB $APP
aws ec2 wait instance-terminated --instance-ids $WEB $APP

aws ec2 delete-security-group --group-id $SG_APP     # first: it references SG-web
aws ec2 delete-security-group --group-id $SG_WEB

aws ec2 disassociate-route-table --association-id $ASSOC
aws ec2 delete-route-table --route-table-id $RT

aws ec2 detach-internet-gateway --internet-gateway-id $IGW --vpc-id $VPC
aws ec2 delete-internet-gateway --internet-gateway-id $IGW

aws ec2 delete-subnet --subnet-id $PUB
aws ec2 delete-subnet --subnet-id $PRIV
aws ec2 delete-vpc --vpc-id $VPC

aws ec2 describe-vpcs --filters Name=tag:Name,Values=lab08-vpc \
  --query 'Vpcs[].VpcId' --output text      # empty
```

The main route table and the default security group of the VPC are deleted together with the VPC. Then click **End Lab**.

Track B — no AWS (Linux network namespaces)

Linux only: an Ubuntu VM or WSL2 on Windows, with `sudo`. It does **not** work on macOS. A network namespace is an isolated network stack, like a tiny machine with its own interfaces and routes.

AWS (Track A)	Linux (Track B)
VPC `10.0.0.0/16`	The host acting as the VPC router
Subnet	A bridge: `br-pub` (`10.0.1.0/24`), `br-priv` (`10.0.2.0/24`)
Instance	Network namespace: `web` , `app`
Network interface	veth pair (one end in the namespace, one on the bridge)
Route table	`ip route` inside each namespace
Local route	Host forwards between the two bridges
Internet gateway + public IP	NAT (`MASQUERADE`) for `10.0.1.0/24` only
Security group	`iptables` rules inside the `app` namespace

```
sudo apt install -y iproute2 iptables iputils-ping curl python3
sudo iptables-save > ~/before-lab08.rules     # snapshot, for cleanup
```

B1. Two "subnets" and two "instances"

```
sudo sysctl -w net.ipv4.ip_forward=1
sudo ip netns add web
sudo ip netns add app

# subnets = bridges; the host's address on each is the gateway (.1)
sudo ip link add br-pub type bridge
sudo ip link add br-priv type bridge
sudo ip addr add 10.0.1.1/24 dev br-pub
sudo ip addr add 10.0.2.1/24 dev br-priv
sudo ip link set br-pub up
sudo ip link set br-priv up

# web: interface 10.0.1.10 in the public subnet
sudo ip link add veth-web type veth peer name veth-web-br
sudo ip link set veth-web netns web
sudo ip link set veth-web-br master br-pub up
sudo ip netns exec web ip addr add 10.0.1.10/24 dev veth-web
sudo ip netns exec web ip link set veth-web up
sudo ip netns exec web ip link set lo up
sudo ip netns exec web ip route add default via 10.0.1.1

# app: interface 10.0.2.10 in the private subnet
sudo ip link add veth-app type veth peer name veth-app-br
sudo ip link set veth-app netns app
sudo ip link set veth-app-br master br-priv up
sudo ip netns exec app ip addr add 10.0.2.10/24 dev veth-app
sudo ip netns exec app ip link set veth-app up
sudo ip netns exec app ip link set lo up
sudo ip netns exec app ip route add default via 10.0.2.1
```

B2. Routing: local route and "internet gateway"

```
UPLINK=$(ip route show default | awk '{print $5; exit}')
echo $UPLINK                                    # e.g. eth0

# private subnet: no way to the internet
sudo iptables -I FORWARD 1 -s 10.0.2.0/24 ! -d 10.0.0.0/16 -j REJECT
# "local route": the two subnets can talk
sudo iptables -I FORWARD 2 -s 10.0.0.0/16 -d 10.0.0.0/16 -j ACCEPT
# public subnet: out to the internet, replies back in
sudo iptables -I FORWARD 3 -s 10.0.1.0/24 -o $UPLINK -j ACCEPT
sudo iptables -I FORWARD 4 -d 10.0.1.0/24 -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
sudo iptables -t nat -A POSTROUTING -s 10.0.1.0/24 -o $UPLINK -j MASQUERADE
```

B3. A "security group" for app

Run a small service in `app`, then allow it only from `web`:

```
sudo ip netns exec app python3 -m http.server 8080 --bind 10.0.2.10 &

sudo ip netns exec app iptables -A INPUT -i lo -j ACCEPT
sudo ip netns exec app iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
sudo ip netns exec app iptables -A INPUT -s 10.0.1.10 -p tcp --dport 8080 -j ACCEPT
sudo ip netns exec app iptables -A INPUT -s 10.0.1.10 -p icmp -j ACCEPT
sudo ip netns exec app iptables -P INPUT DROP
```

B4. Prove it

```
sudo ip netns exec web ping -c 2 10.0.2.10               # works
sudo ip netns exec web curl -m 5 http://10.0.2.10:8080   # works (allowed by app's rules)
curl -m 5 http://10.0.2.10:8080                          # from the host: times out (dropped)

sudo ip netns exec web ping -c 2 1.1.1.1                 # works: public subnet has NAT
sudo ip netns exec app ping -c 2 1.1.1.1                 # fails: private subnet has no way out
```

Use IP addresses (`1.1.1.1`), not names: DNS inside a namespace may not work, depending on how your system resolves names.

B5. Break it, then fix it

```
sudo iptables -t nat -D POSTROUTING -s 10.0.1.0/24 -o $UPLINK -j MASQUERADE
sudo ip netns exec web ping -c 2 -W 2 1.1.1.1            # fails now
sudo iptables -t nat -A POSTROUTING -s 10.0.1.0/24 -o $UPLINK -j MASQUERADE
sudo ip netns exec web ping -c 2 1.1.1.1                 # works again
```

B6. Cleanup

```
sudo pkill -f "http.server 8080"
sudo ip netns del web
sudo ip netns del app
sudo ip link del br-pub
sudo ip link del br-priv
sudo iptables-restore < ~/before-lab08.rules
```

Be ready to explain

1. What makes your public subnet public, and your private subnet private?
2. The app server is in the same VPC as the web server. Why can the web server reach it, but your laptop can't?
3. In A7 the security group still allowed port 80. Why did the page stop loading when you removed the route?
4. Why does SG-app use `--source-group $SG_WEB` instead of the web server's IP address?
5. The app server needs to install updates. What would you add, where, and what does it cost to leave running?
6. Why did the cleanup have to delete SG-app before SG-web, and the IGW before the VPC?

Common mistakes

- **Web page times out.** Check, in order: the route table has `0.0.0.0/0 → igw` and is **associated with the public subnet**, the instance has a public IP (did you run `modify-subnet-attribute` **before** launching?), SG-web allows 80, and cloud-init has finished.
- **Web instance has no public IP.** It was launched before `--map-public-ip-on-launch` was set. Terminate it and launch again.
- **SSH from web to app hangs.** SG-app doesn't allow 22 from SG-web, or you typed the wrong address. The app server has no public IP: use its private `10.0.2.x` address.
- **`Permission denied (publickey)` on the second hop.** The key isn't on the web server: you forgot `ssh-add` + `ssh -A`, or the `scp` fallback.
- **`DependencyViolation` during cleanup.** Something still uses the resource: instances not fully terminated, SG-app still referencing SG-web, or the IGW still attached. Follow the order in A8.
- **`InvalidClientTokenId` / `ExpiredToken`.** The Learner Lab session ended. Start Lab again, copy fresh credentials, and look up your IDs (`describe-vpcs --filters Name=tag:Name,Values=lab08-vpc`).
- **Track B: everything is blocked.** Docker or a firewall on the host set `FORWARD` to `DROP`. The rules above are inserted at the top with `-I`, so check `sudo iptables -L FORWARD -n --line-numbers` for their order.